

Interface Analysis

This chapter describes the timing analysis procedures for various types of input and output paths, and several commonly used interfaces. Timing analysis of special interfaces such as for SRAMs and the timing analysis of source synchronous interfaces such as those used for DDR SDRAMs are also described.

9.1 IO Interfaces

This section presents examples that illustrate how the constraints on input and output interfaces of the DUA are defined. Later sections provide examples of timing constraints for the SRAM and DDR SDRAM interfaces.

9.1.1 Input Interface

There are broadly two alternate ways of specifying the timing of the inputs:

- i. The waveforms at an input of the DUA are provided as AC specifications¹.
- ii. The path delay of the external logic to an input is specified.

Waveform Specification at Inputs

Consider the input AC specification shown in Figure 9-1. The specification is that the input *CIN* is stable 4.3ns before the rising edge of clock *CLKP*, and that the value remains stable until 2ns after the rising edge of the clock.

Consider the 4.3ns specification first. Given the clock cycle of 8ns (as shown in Figure 9-1), this requirement maps into the delay from the virtual flip-flop (the flip-flop that is driving this input) to the input *CIN*. The delay from the virtual flip-flop clock to *CIN* must be at most 3.7ns ($= 8.0 - 4.3$), the maximum delay being at 3.7ns. This ensures that the data at input *CIN* arrives 4.3ns prior to the rising edge. Hence, this part of the AC specification can equivalently be specified as a max input delay of 3.7ns.

The AC specification also states that the input *CIN* is stable for 2ns after the rising edge of the clock. This specification can also be mapped into the delay from the virtual flip-flop, that is, the delay from the virtual flip-flop to input *CIN* must be at least 2.0ns. Hence the minimum input delay is specified as 2.0ns.

1. Specification of a digital device is provided in two parts: DC - the constant values (static), and AC - the changing waveforms (dynamic).

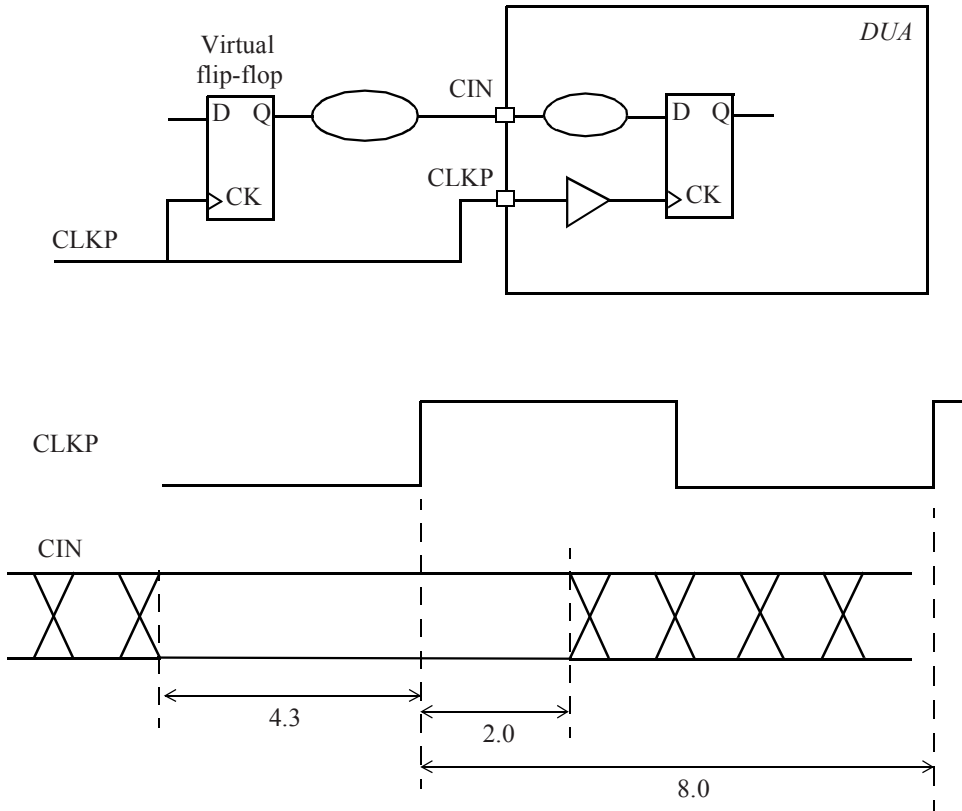


Figure 9-1 *AC specification for input port.*

Here are the input constraints.

```
create_clock -name CLKP -period 8 [get_ports CLKP]
set_input_delay -min 2.0 -clock CLKP [get_ports CIN]
set_input_delay -max 3.7 -clock CLKP [get_ports CIN]
```

Here are the path reports for the design under these input conditions. First is the setup report.

Startpoint: CIN (input port clocked by CLKP)
Endpoint: UFF4 (rising edge-triggered flip-flop clocked by CLKP)
Path Group: CLKP
Path Type: max

Point	Incr	Path
-----	-----	-----
clock CLKP (rise edge)	0.00	0.00
clock network delay (propagated)	0.00	0.00
input external delay	3.70	3.70 f
CIN (in)	0.00	3.70 f
UBUF5/Z (BUFF)	0.05	3.75 f
UXOR1/Z (XOR2)	0.10	3.85 r
UFF4/D (DF)	0.00	3.85 r
data arrival time		3.85
clock CLKP (rise edge)	8.00	8.00
clock source latency	0.00	8.00
CLKP (in)	0.00	8.00 r
UCKBUF4/C (CKB)	0.07	8.07 r
UCKBUF5/C (CKB)	0.06	8.13 r
UFF4/CP (DF)	0.00	8.13 r
library setup time	-0.05	8.08
data required time		8.08
-----		-----
data required time		8.08
data arrival time		-3.85
-----		-----
slack (MET)		4.23

The max input delay specified (3.7ns) is added to the data path. The setup check ensures that delay inside the DUA is less than 4.3ns and the proper data can be latched. Next is the hold timing report.

Startpoint: CIN (input port clocked by CLKP)
Endpoint: UFF4 (rising edge-triggered flip-flop clocked by CLKP)

Path Group: CLKP
Path Type: min

Point	Incr	Path
-----		-----
clock CLKP (rise edge)	0.00	0.00
clock network delay (propagated)	0.00	0.00
input external delay	2.00	2.00 r
CIN (in) <-	0.00	2.00 r
UBUF5/Z (BUFF)	0.05	2.05 r
UXOR1/Z (XOR2)	0.07	2.12 r
UFF4/D (DF)	0.00	2.12 r
data arrival time		2.12
clock CLKP (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKP (in)	0.00	0.00 r
UCKBUF4/C (CKB)	0.07	0.07 r
UCKBUF5/C (CKB)	0.06	0.13 r
UFF4/CK (DF)	0.00	0.13 r
clock uncertainty	0.05	0.18
library hold time	-0.00	0.17
data required time		0.17
-----		-----
data required time		0.17
data arrival time		-2.12
-----		-----
slack (MET)		1.95

The min input delay is added to the data path in the hold check. The check ensures that the earliest data change at 2ns after the clock edge does not overwrite the previous data at the flip-flop.

Path Delay Specification to Inputs

When the path delays of the external logic connected to an input are known, specifying the input constraints is a straightforward task. Any delays along the external logic path to the input are added and the path delay is specified using the *set_input_delay* command.

Figure 9-2 shows an example of external logic path to input. The T_{ck2q} and T_{c1} delays are added to obtain the external delay. Knowing T_{ck2q} and T_{c1} , the input delay is directly obtained as $T_{ck2q} + T_{c1}$.

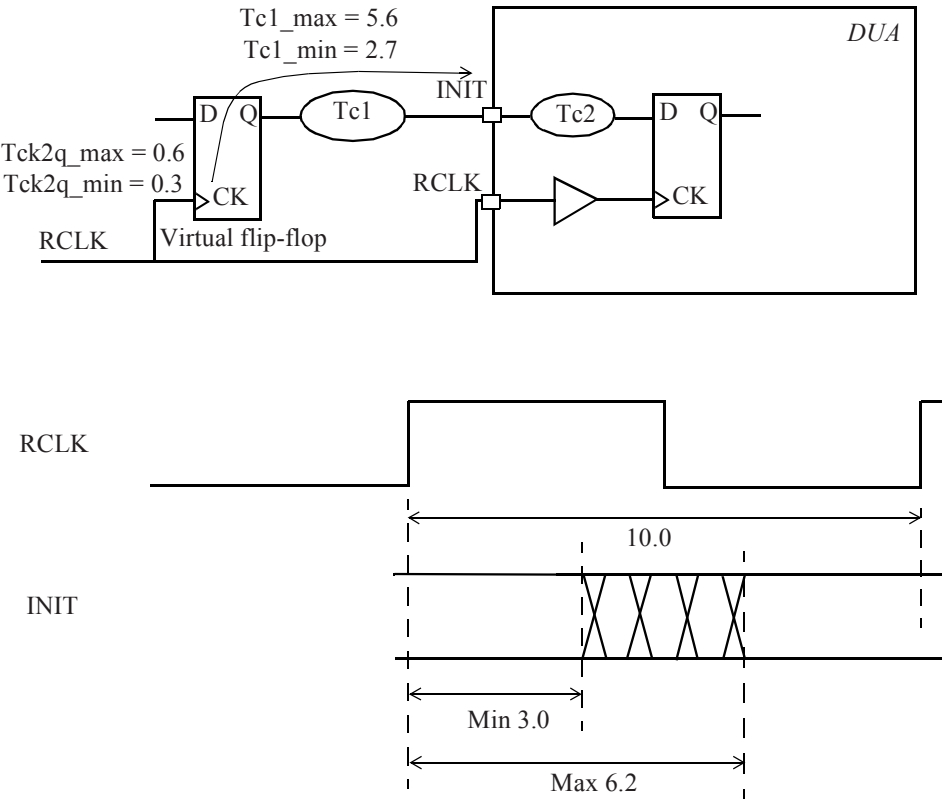


Figure 9-2 *Input path delay specifications.*

The external max and min path delays translate to the following input constraints.

```
create_clock -name RCLK -period 10 [get_ports RCLK]
set_input_delay -max 6.2 -clock RCLK [get_ports INIT]
set_input_delay -min 3.0 -clock RCLK [get_ports INIT]
```

The path reports for these are similar to the ones in Section 8.1 and Section 8.2.

Note that when computing the arrival time at the data pin of the flip-flop inside the design, the max and min input delay values get added to the data path delay depending on whether a *max path check* (setup) or a *min path check* (hold) is being performed.

9.1.2 Output Interface

Similar to the input case, there are broadly two alternate ways for specifying the output timing requirements:

- i. The required waveforms at the output of the DUA are provided as AC specifications.
- ii. The path delay of external logic is specified.

Output Waveform Specification

Consider the output AC specification shown in Figure 9-3. The output *QOUT* should be stable at the output 2ns prior to the rising edge of clock *CLKP*. Also, the output should not change until 1.5ns after the rising edge of the clock. These constraints are normally obtained from the setup and hold requirements of the external block that interfaces with *QOUT*.

Here are the constraints that expresses this requirement on the output.

```
create_clock -name CLKP -period 6 \
-waveform {0 3} [get_ports CLKP]
```

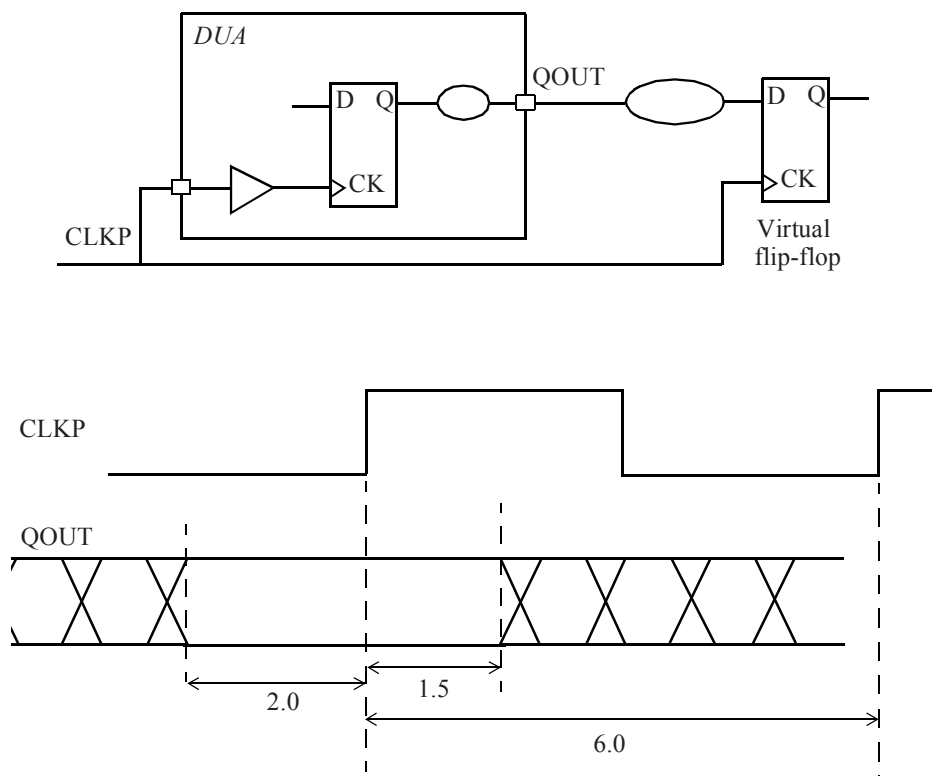


Figure 9-3 Output AC specification.

```
# Setup delay of virtual flip-flop:
set_output_delay -clock CLKP -max 2.0 [get_ports QOUT]
# Hold time for virtual flip-flop:
set_output_delay -clock CLKP -min -1.5 [get_ports QOUT]
```

The maximum output path delay is specified as 2.0ns. This will ensure that data *QOUT* changes ahead of the 2ns window before the clock edge. The minimum output path delay of -1.5ns specifies the requirement from the perspective of the virtual flip-flop, that is, to ensure the 1.5ns hold require-

ment at the output *QOUT*. A hold requirement of 1.5ns maps to a *set_output_delay* min of -1.5ns.

Here is the setup timing path report.

Startpoint: UFF6 (rising edge-triggered flip-flop clocked by CLKP)
 Endpoint: QOUT (output port clocked by CLKP)
 Path Group: CLKP
 Path Type: max

Point	Incr	Path

clock CLKP (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKP (in)	0.00	0.00 r
UCKBUF4/C (CKB)	0.07	0.07 r
UCKBUF6/C (CKB)	0.07	0.14 r
UCKBUF7/C (CKB)	0.05	0.19 r
UFF6/CK (DFCN)	0.00	0.19 r
UFF6/Q (DFCN)	0.16	0.35 r
UAND1/Z (AN2)	1.31	1.66 r
QOUT (out)	0.00	1.66 r
data arrival time		1.66
clock CLKP (rise edge)	6.00	6.00
clock network delay (propagated)	0.00	6.00
clock uncertainty	-0.30	5.70
output external delay	-2.00	3.70
data required time		3.70

data required time		3.70
data arrival time		-1.66

slack (MET)		2.04

The max output delay is subtracted from the next clock edge to determine the required arrival time at the output of the DUA.

The hold timing check path report is next.

Startpoint: UFF4 (rising edge-triggered flip-flop clocked by CLKP)
Endpoint: QOUT (output port clocked by CLKP)
Path Group: CLKP
Path Type: min

Point	Incr	Path

clock CLKP (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKP (in)	0.00	0.00 r
UCKBUF4/C (CKB)	0.07	0.07 r
UCKBUF5/C (CKB)	0.06	0.13 r
UFF4/CK (DF)	0.00	0.13 r
UFF4/Q (DF)	0.14	0.27 f
UAND1/Z (AN2)	0.75	1.02 f
QOUT (out)	0.00	1.02 f
data arrival time		1.02
clock CLKP (rise edge)	0.00	0.00
clock network delay (propagated)	0.00	0.00
clock uncertainty	0.05	0.05
output external delay	1.50	1.55
data required time		1.55

data required time		1.55
data arrival time		-1.02

slack (VIOLATED)		-0.53

The min output delay (of -1.5ns) is subtracted from the capture clock edge to determine the earliest arrival time at the output of the DUA that meets the hold requirement. It is common to have a negative min output delay requirement.

External Path Delays for Output

In this case, the path delay of the external logic is explicitly specified. See the example in Figure 9-4.

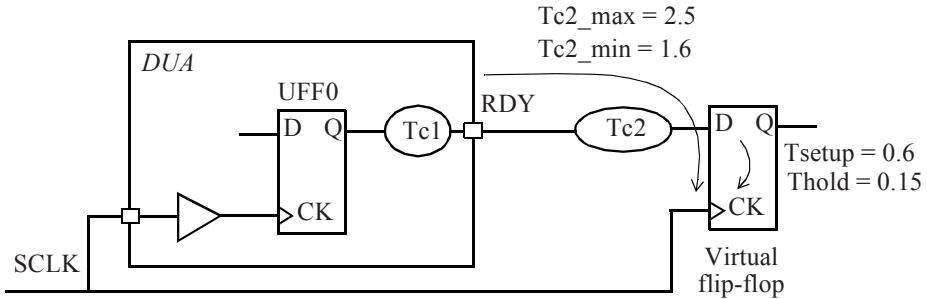


Figure 9-4 Output path delay specifications.

Let us examine the setup check first. A max output delay (*set_output_delay - max*) setting is obtained from the *Tc2_max* and *Tsetup*. To check the setup requirement for output paths between flip-flops inside DUA (such as UFF0) and the virtual flip-flop, the max output delay is specified as $Tc2_max + Tsetup$.

Next let us examine the hold check. A min output delay (*set_output_delay - min*) setting is obtained from *Tc2_min* and *Thold*. Since the hold of a capture flip-flop is added to the capture clock path, the min output delay is specified as $(Tc2_min - Thold)$.

The constraints on the output translate to the following:

```
create_clock -name SCLK -period 5 [get_ports SCLK]
# Setup of the external logic (Tc2_max = 2.5,
# Tsetup = 0.6):
set_output_delay -max 3.1 -clock SCLK [get_ports RDY]
# Hold of the external logic (Tc2_min=1.6, Thold=0.15):
set_output_delay -min 1.45 -clock SCLK [get_ports RDY]
```

The path reports for these are similar to the ones in Section 8.1 and Section 8.2.

9.1.3 Output Change within Window

The *set_output_delay* specification can be used to specify maximum and minimum arrival times of an output signal with respect to a clock. This section considers the special case of specifying constraints that verify the scenario when the output can change only within a timing window relative to the clock edge. This requirement occurs quite often while verifying the timing of source synchronous interfaces.

In source synchronous interfaces, the clock also appears along with the data as an output. In such cases, there is normally a requirement for a timing relationship between the clock and the data. For example, the output data may be required to change only within a specific window around the rising edge of the clock.

An example requirement for a source synchronous interface is shown in Figure 9-5.

The requirement is that each bit of *DATAQ* can only change in the specified window 2ns prior to the clock rising edge and up to 1ns after the clock rising edge. This is quite different from the output delay specifications discussed in previous sections where the data pins are required to be stable in a specified timing window around the clock rising edge.

We create a generated clock on *CLK_STROBE* whose master clock is *CLKM*. This is to help specify timing constraints corresponding to the requirements of this interface.

```
create_clock -name CLKM -period 6 [get_ports CLKM]
create_generated_clock -name CLK_STROBE -source CLKM \
    -divide_by 1 [get_ports CLK_STROBE]
```

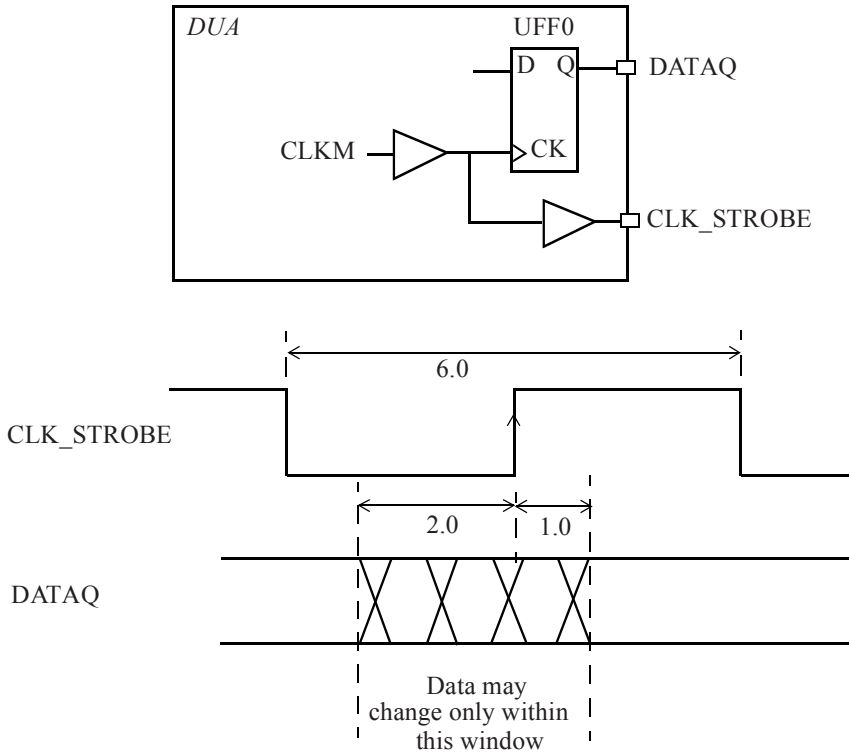


Figure 9-5 *Data change allowed only within a window around the clock.*

The window requirement is specified using a combination of setup and hold checks with multicycle path specifications. The timing requirement is mapped to a setup check that has to occur on a single rising edge (same edge for launch and capture). Thus, we specify a multicycle of 0 for setup.

```
set_multicycle_path 0 -setup -to [get_ports DATAQ]
```

In addition, the hold check has to occur on the same edge, thus we need to specify a multicycle of -1 (minus one) for hold check.

```
set_multicycle_path -1 -hold -to [get_ports DATAQ]
```

Now specify the timing constraints on the output with respect to the clock *CLK_STROBE*.

```
set_output_delay -max -1.0 -clock CLK_STROBE \  
[get_ports DATAQ]  
set_output_delay -min +2.0 -clock CLK_STROBE \  
[get_ports DATAQ]
```

Notice that the *min* value of the output delay specification is larger than the *max* specification. This anomaly exists because, in this scenario, the output delay specification does not correspond to an actual logic block. Unlike the case of a typical output interface where the output delay specification corresponds to a logic block at the output, the *set_output_delay* specification in a source synchronous interface is just a mechanism to verify whether the outputs are constrained to switch within a specified window around the clock. Thus, we have the anomaly of the *min* output delay specification being larger than the *max* output delay specification.

Here is the setup timing check path report for the constraints specified.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLKM)
Endpoint: DATAQ (output port clocked by CLK_STROBE)
Path Group: CLK_STROBE
Path Type: **max**

Point	Incr	Path

clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UINV0/ZN (INV)	0.01	0.01 f
UINV1/ZN (INV)	0.03	0.04 r
UCKBUF0/C (CKB)	0.06	0.10 r
UFF0/CK (DF)	0.00	0.10 r
UFF0/Q (DF)	0.13	0.23 r
UBUF1/Z (BUFF)	0.38	0.61 r

DATAQ (out)	0.00	0.61 r
data arrival time		0.61
clock CLK_STROBE (rise edge)	0.00	0.00
clock CLKM (source latency)	0.00	0.00
CLKM (in)	0.00	0.00 r
UINV0/ZN (INV)	0.01	0.01 f
UINV1/ZN (INV)	0.03	0.04 r
UCKBUF1/C (CKB)	0.05	0.09 r
CLK_STROBE (out)	0.00	0.09 r
clock uncertainty	-0.30	-0.21
output external delay	1.00	0.79
data required time		0.79

data required time		0.79
data arrival time		-0.61

slack (MET)		0.18

Notice that the launch and the capture edges are the same clock edge, which is at time 0. The report shows that *DATAQ* changes at 0.61ns while the *CLK_STROBE* changes at 0.09ns. Since *DATAQ* can change within 1ns of the *CLK_STROBE*, there is a slack of 0.18ns after accounting for the 0.3ns of clock uncertainty.

Here is the hold path report that checks for the bound on the other side of the clock.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLKM)
 Endpoint: DATAQ (output port clocked by CLK_STROBE)
 Path Group: CLK_STROBE
 Path Type: **min**

Point	Incr	Path

clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UINV0/ZN (INV)	0.01	0.01 f
UINV1/ZN (INV)	0.03	0.04 r

UCKBUF0/C (CKB)	0.06	0.10 r
UFF0/CK (DF)	0.00	0.10 r
UFF0/Q (DF)	0.13	0.23 f
UBUF1/Z (BUFF)	0.25	0.48 f
DATAQ (out)	0.00	0.48 f
data arrival time		0.48
clock CLK_STROBE (rise edge)	0.00	0.00
clock CLKM (source latency)	0.00	0.00
CLKM (in)	0.00	0.00 r
UINV0/ZN (INV)	0.01	0.01 f
UINV1/ZN (INV)	0.03	0.04 r
UCKBUF1/C (CKB)	0.05	0.09 r
CLK_STROBE (out)	0.00	0.09 r
clock uncertainty	0.05	0.14
output external delay	-2.00	-1.86
data required time		-1.86

data required time		-1.86
data arrival time		-0.48

slack (MET)		2.35

With the min path analysis, *DATAQ* arrives at 0.48ns while the *CLK_STROBE* arrives at 0.09ns. Since the requirement is that data can change up to a 2ns limit before *CLK_STROBE*, we get a slack of 2.35ns after accounting for the clock uncertainty of 50ps.

Another example of a source synchronous interface is depicted in Figure 9-6. In this case, the output clock is a divide-by-2 of the main clock and is part of the synchronous interface with the data. The *POUT* is constrained to switch no earlier than 2ns before and no later than 1ns after the *QCLKOUT*.

Here are the constraints.

```
create_clock -name CLKM -period 6 [get_ports CLKM]
create_generated_clock -name QCLKOUT -source CLKM \
    -divide_by 2 [get_ports QCLKOUT]

set_multicycle_path 0 -setup -to [get_ports POUT]
```

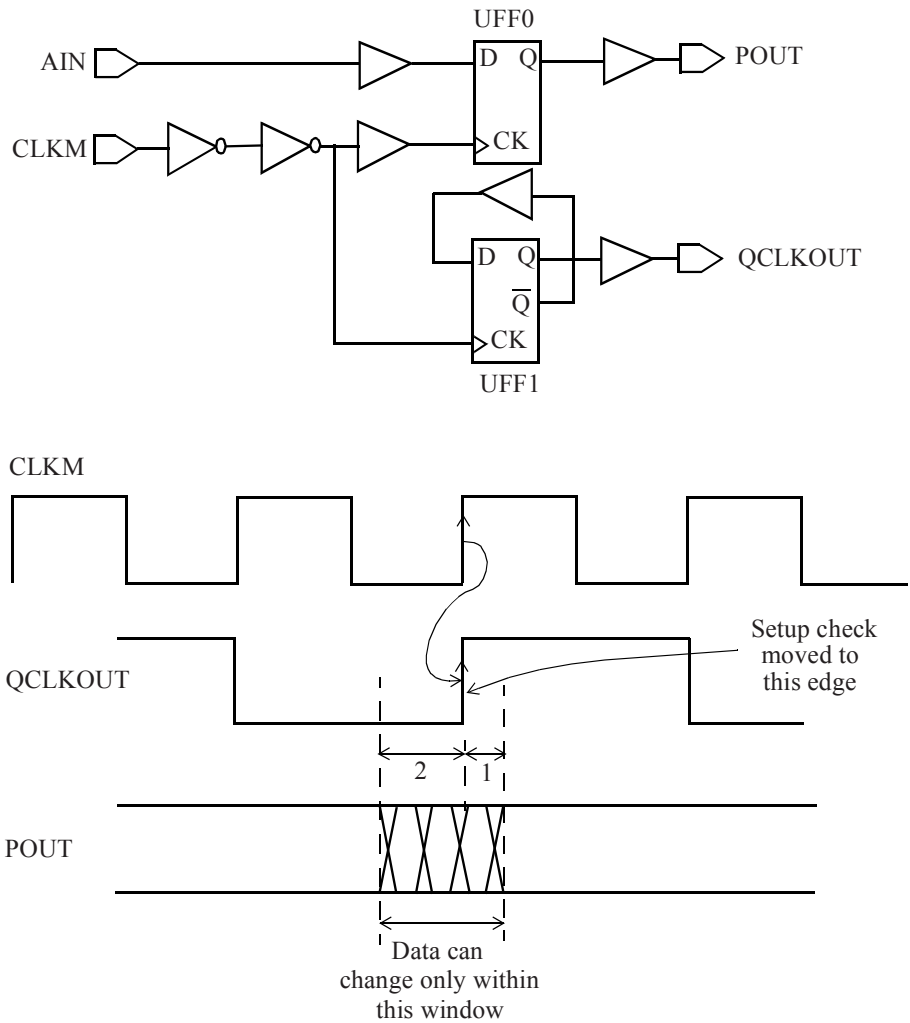


Figure 9-6 Data change allowed only around divide-by-2 output clock.

```
set_multicycle_path -1 -hold -to [get_ports POUT]

set_output_delay -max -1.0 -clock QCLKOUT [get_ports POUT]
set_output_delay -min +2.0 -clock QCLKOUT [get_ports POUT]
```

Here is the setup timing report.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLKM)
Endpoint: POUT (output port clocked by QCLKOUT)
Path Group: QCLKOUT
Path Type: **max**

Point	Incr	Path

clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UINV0/ZN (INV)	0.01	0.01 f
UINV1/ZN (INV)	0.03	0.04 r
UCKBUF0/C (CKB)	0.06	0.10 r
UFF0/CK (DF)	0.00	0.10 r
UFF0/Q (DF)	0.13	0.23 r
UBUF1/Z (BUFF)	0.38	0.61 r
POUT (out)	0.00	0.61 r
data arrival time		0.61
clock QCLKOUT (rise edge)	0.00	0.00
clock CLKM (source latency)	0.00	0.00
CLKM (in)	0.00	0.00 r
UINV0/ZN (INV)	0.01	0.01 f
UINV1/ZN (INV)	0.03	0.04 r
UCKBUF1/C (CKB)	0.06	0.10 r
UFF1/Q (DF)	0.13	0.23 r
UBUF2/Z (BUFF)	0.04	0.27 r
QCLKOUT (out)	0.00	0.27 r
clock uncertainty	-0.30	-0.03
output external delay	1.00	0.97
data required time		0.97

data required time		0.97
data arrival time		-0.61

 slack (MET)

0.36

Notice that the multicycle specification has moved the setup check one cycle back so that the check is performed on the same clock edge. Output *POUT* changes at 0.61ns while the clock *QCLKOUT* changes at 0.27ns. Given the requirement of changing within 1ns, and considering the clock uncertainty of 0.30ns, we get a slack of 0.36ns.

Here is the hold path report that checks for the other constraint on the switching window.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLKM)
 Endpoint: POUT (output port clocked by QCLKOUT)
 Path Group: QCLKOUT
 Path Type: **min**

Point	Incr	Path
clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UINV0/ZN (INV)	0.01	0.01 f
UINV1/ZN (INV)	0.03	0.04 r
UCKBUF0/C (CKB)	0.06	0.10 r
UFF0/CK (DF)	0.00	0.10 r
UFF0/Q (DF)	0.13	0.23 f
UBUF1/Z (BUFF)	0.25	0.48 f
POUT (out)	0.00	0.48 f
data arrival time		0.48
clock QCLKOUT (rise edge)	0.00	0.00
clock CLKM (source latency)	0.00	0.00
CLKM (in)	0.00	0.00 r
UINV0/ZN (INV)	0.01	0.01 f
UINV1/ZN (INV)	0.03	0.04 r
UCKBUF1/C (CKB)	0.06	0.10 r
UFF1/Q (DF)	0.13	0.23 r
UBUF2/Z (BUFF)	0.04	0.27 r
QCLKOUT (out)	0.00	0.27 r

clock uncertainty	0.05	0.32
output external delay	-2.00	-1.68
data required time		-1.68

data required time		-1.68
data arrival time		-0.48

slack (MET)		2.17

The path report shows that the data changes within the allowable window of 2ns before the *QCLKOUT* clock edge and there is a slack of 2.17ns.

9.2 SRAM Interface

All data transfers in an SRAM interface occur only on the active edge of the clock. All signals are latched by the SRAM or launched by the SRAM on the active clock edge only. The signals comprising the SRAM interface include the command, address and control output bus (CAC), the bidirectional data bus (DQ) and the clock. In the write cycle, the DUA writes data to the SRAM, the data and address go from the DUA to the SRAM, and are latched in the SRAM on the active clock edge. In the read cycle, the address signals still go from the DUA to the SRAM while the data signals output from the SRAM go to the DUA. The address and control are thus unidirectional and go from the DUA to the SRAM as shown in Figure 9-7. A DLL (delay-locked loop¹) is typically placed in the clock path. The DLL allows the clock to be delayed, if necessary, to account for delay variations of the various signals across the interface due to PVT tolerances and other external variations. By accounting for such variations, there is a good timing margin for the data transfer for both the read cycle and the write cycle to and from the SRAM.

Figure 9-8 shows the AC characteristics of a typical SRAM interface. Note that the *Data in* and *Data out* in Figure 9-8 refer to the direction seen by the

1. See [BES07] in Bibliography.

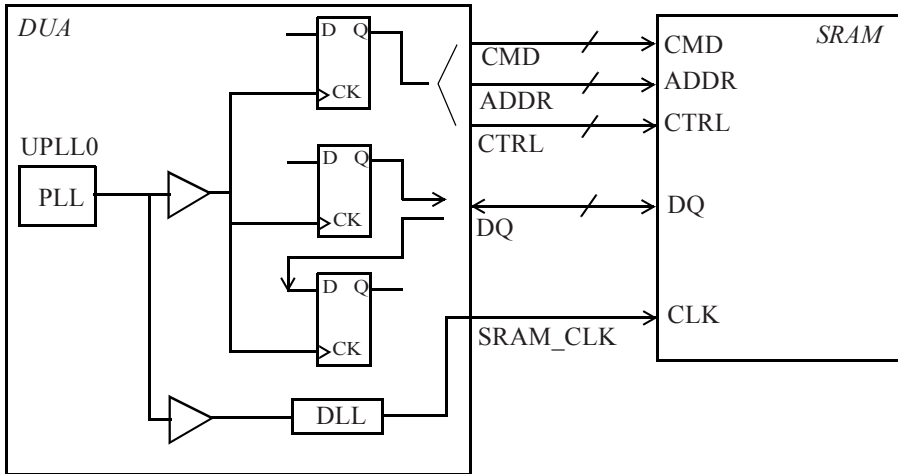


Figure 9-7 *SRAM interface.*

SRAM; the *Data out* from the SRAM is the input to the DUA and the *Data in* to the SRAM is the output of the DUA. These requirements translate to the following IO interface constraints for the DUA interfacing the SRAM.

```
# First define primary clock at the output of UPLL0:
create_clock -name PLL_CLK -period 5 [get_pins UPLL0/CLKOUT]
# Next define a generated clock at clock output pin of DUA:
create_generated_clock -name SRAM_CLK \
  -source [get_pins UPLL0/CLKOUT] -divide_by 1 \
  [get_ports SRAM_CLK]
# Constrain the address and control:
set_output_delay -max 1.5 -clock SRAM_CLK \
  [get_ports ADDR[0]]
set_output_delay -min -0.5 -clock SRAM_CLK \
  [get_ports ADDR[0]]
. . .
```

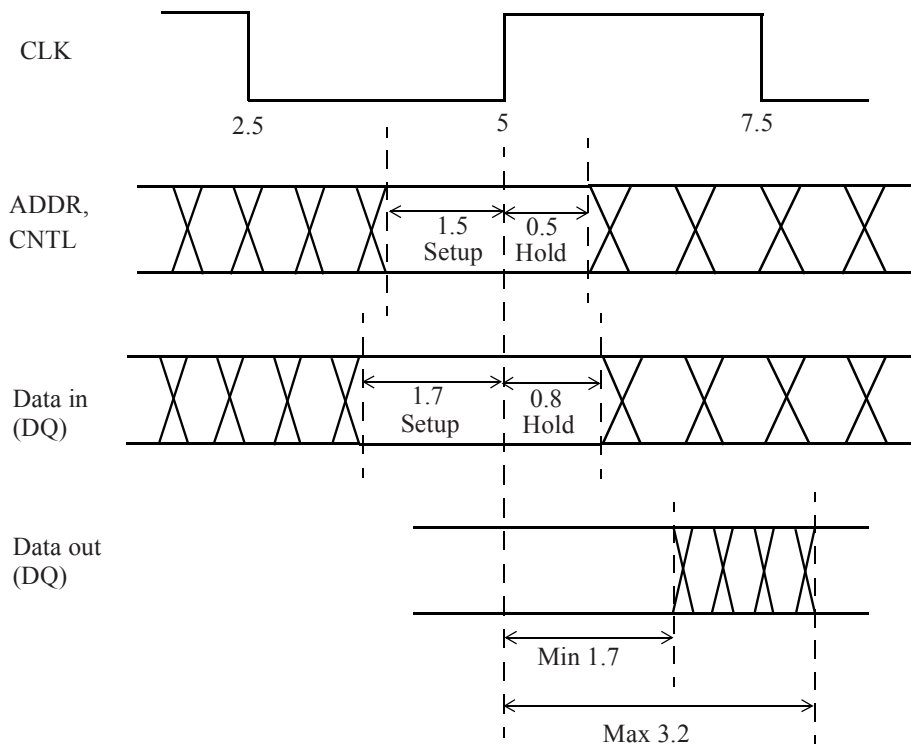


Figure 9-8 *SRAM AC characteristics.*

```
# Constrain the data going out of DUA:
set_output_delay -max 1.7 -clock SRAM_CLK [get_ports DQ[0]]
set_output_delay -min -0.8 -clock SRAM_CLK [get_ports DQ[0]]
# Constrain the data coming into the DUA:
set_input_delay -max 3.2 -clock SRAM_CLK [get_ports DQ[0]]
set_input_delay -min 1.7 -clock SRAM_CLK [get_ports DQ[0]]
. . .
```

Here is a representative setup path report for the address pin.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLKM)

Endpoint: ADDR (output port clocked by SRAM_CLK)

Path Group: SRAM_CLK

Path Type: **max**

Point	Incr	Path

clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UINV0/ZN (INV)	0.01	0.01 f
UINV1/ZN (INV)	0.04	0.05 r
UCKBUF0/C (CKB)	0.06	0.11 r
UFF0/CP (DF)	0.00	0.11 r
UFF0/Q (DF)	0.13	0.24 f
UBUF1/Z (BUFF)	0.05	0.29 f
ADDR (out)	0.00	0.29 f
data arrival time		0.29
clock SRAM_CLK (rise edge)	10.00	10.00
clock CLKM (source latency)	0.00	10.00
CLKM (in)	0.00	10.00 r
UINV0/ZN (INV)	0.01	10.01 f
UINV1/ZN (INV)	0.04	10.05 r
UCKBUF2/C (CKB)	0.05	10.10 r
SRAM_CLK (out)	0.00	10.10 r
clock uncertainty	-0.30	9.80
output external delay	-1.50	8.30
data required time		8.30

data required time		8.30
data arrival time		-0.29

slack (MET)		8.01

The setup check validates whether the address signal arrives at the memory 1.5ns (setup time of address pin of memory) prior to the *SRAM_CLK* edge.

Here is the hold timing path report for the same pin.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLKM)
Endpoint: ADDR (output port clocked by SRAM_CLK)
Path Group: SRAM_CLK
Path Type: **min**

Point	Incr	Path

clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UINV0/ZN (INV)	0.01	0.01 f
UINV1/ZN (INV)	0.04	0.05 r
UCKBUF0/C (CKB)	0.06	0.11 r
UFF0/CP (DF)	0.00	0.11 r
UFF0/Q (DF)	0.13	0.24 r
UBUF1/Z (BUFF)	0.04	0.28 r
ADDR (out)	0.00	0.28 r
data arrival time		0.28
clock SRAM_CLK (rise edge)	0.00	0.00
clock CLKM (source latency)	0.00	0.00
CLKM (in)	0.00	0.00 r
UINV0/ZN (INV)	0.01	0.01 f
UINV1/ZN (INV)	0.04	0.05 r
UCKBUF2/C (CKB)	0.05	0.10 r
SRAM_CLK (out)	0.00	0.10 r
clock uncertainty	0.05	0.15
output external delay	0.50	0.65
data required time		0.65

data required time		0.65
data arrival time		-0.28

slack (VIOLATED)		-0.37

The hold checks validate whether the address remains stable for 0.5ns after the clock edge.

9.3 DDR SDRAM Interface

The DDR SDRAM interface can be considered as an extension of the SRAM interface described in the previous section. Just like the SRAM interface, there are two main buses. Figure 9-9 illustrates the bus connectivity and the bus directions between the DUA and the SDRAM. The first bus, which consists of command, address, and control pins (often called CAC) uses the standard scheme of sending information out on one clock edge of a memory clock (or once per clock cycle). The two bidirectional buses consist of the *DQ*, the data bus, and *DQS*, the data strobe. The main differentiation of the DDR interface is the bidirectional data strobe *DQS*. A *DQS* strobe is provided for a group of data signals. This allows the data signals (one per byte or one per half-byte) to have tightly matched timing with the strobe; such a tight match may not be feasible with the clock signal if the clock is common for the entire data bus. The bidirectional strobe signal *DQS* is used for both the read and the write operations. The strobe is used to capture the data on both its edges (both falling and rising edges or Double Data Rate). The *DQ* bus is source synchronous to the data strobe *DQS* (instead of a memory clock) during the read mode of SDRAM, that is, the *DQ* and *DQS* are aligned with each other when they are sent out from the SDRAM. In the other direction, that is when the DUA is sending the data, the *DQS* is phase shifted by 90 degrees. Note that both the data *DQ* and the strobe *DQS* edges are derived from the memory clock inside the DUA.

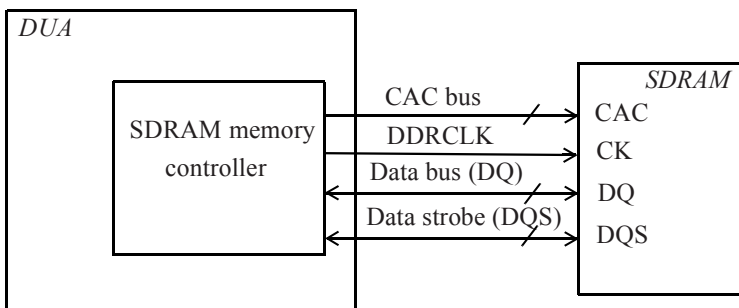


Figure 9-9 *DDR SDRAM interface.*

As described above, there is one data strobe *DQS* for a group of *DQ* signals (four or eight bits). This is done to make the skew balancing requirements between all bits of *DQ* and *DQS* easier. For example, with one *DQS* for a byte, a group of nine signals (eight *DQs* and one *DQS*) needs to be balanced, which is much easier than balancing a 72-bit data bus with the clock.

The above description is not a complete explanation of a DDR SDRAM interface, though sufficient to explain the timing requirements of such an interface.

Figure 9-10 shows the AC characteristics of the CAC bus (at the DUA) for a typical DDR SDRAM interface. These setup and hold requirements map into the following interface constraints for the CAC bus.

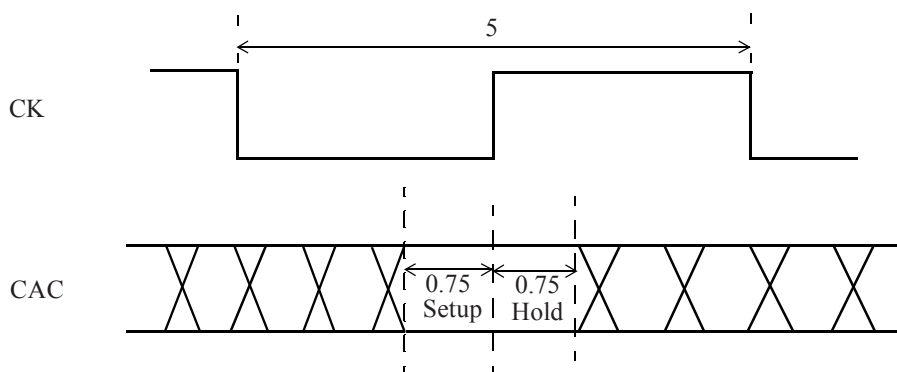


Figure 9-10 AC characteristics of CAC signals for a typical DDR SDRAM interface.

```
# DDRCLK is typically a generated clock of the PLL
# clock internal to DUA:
create_generated_clock -name DDRCLK \
  -source [get_pins UPLL0/CLKOUT] \
  -divide_by 1 [get_ports DDRCLK]
```

```
# Set output constraints for each bit of CAC:  
set_output_delay -max 0.75 -clock DDRCLK [get_ports CAC]  
set_output_delay -min -0.75 -clock DDRCLK [get_ports CAC]
```

The address bus may drive a much greater load than the clock for some scenarios especially when interfacing with unbuffered memory modules. In such cases, the address signals have a larger delay to the memory than the clock signals and this delay difference may result in different AC specifications than the ones depicted in Figure 9-10.

The alignment of *DQS* and *DQ* is different for read and write cycles. This is explored further in the following subsections.

9.3.1 Read Cycle

In a read cycle, the data output from the memory is edge-aligned to *DQS*. Figure 9-11 shows the waveforms; *DQ* and *DQS* in the figure represent the signals at the memory pins. Data (*DQ*) is sent out by the memory on each edge of *DQS* and *DQ* transitions are edge-aligned to the falling and rising edges of *DQS*.

Since the *DQS* strobe signal and the *DQ* data signals are nominally aligned with each other, a DLL (or any alternate method to achieve quarter-cycle delay) is typically used by the memory controller inside the DUA to delay the *DQS* and thus align the delayed *DQS* edge with the center of the data valid window.

Even though the *DQ* and *DQS* are nominally aligned at the memory, the *DQ* and *DQS* strobe signals may no longer be aligned at the memory controller inside the DUA. This can be due to differences in delays between IO buffers and due to factors such as differences in PCB interconnect traces.

Figure 9-12 shows the basic read schematic. The positive edge-triggered flip-flop captures the data *DQ* on the rising edge of *DQS_DLL*, while the negative edge-triggered flip-flop captures the data *DQ* on the falling edge of *DQS_DLL*. While a DLL is not depicted on the *DQ* path, some designs may contain a DLL on the data path also. This allows delaying of the sig-

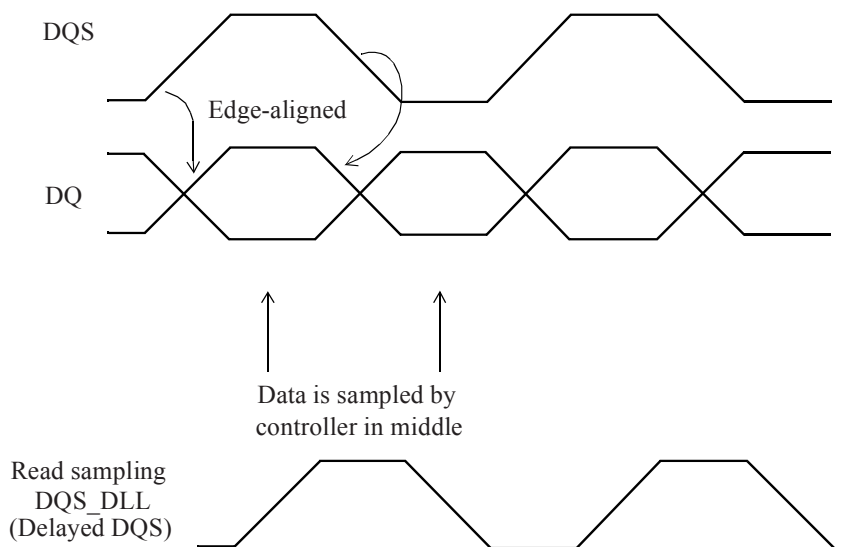


Figure 9-11 *DQS and DQ signals at the memory pins during DDR read cycle.*

nals (to account for variations due to PVT or interconnect trace length or other differences) so that the data can be sampled exactly in the middle of the data valid window.

To constrain the read interface on the controller, a clock is defined on *DQS*, and input delays are specified on the data with respect to the clock.

```
create_clock -period 5 -name DQS [get_ports DQS]
```

This assumes that the memory read interface operates at 200 MHz (equivalent to 400 Mbps as data is transferred on both clock edges), and corresponds to the *DQ* signals being sampled every 2.5ns. Since the data is being captured on both edges, input constraints need to be specified for each edge explicitly.

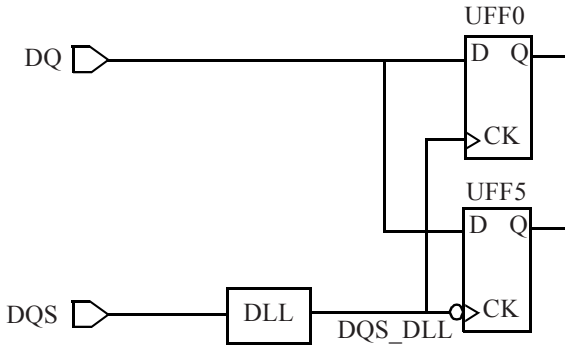


Figure 9-12 *Read capture logic in memory controller.*

```
# For rising clock edge:
set_input_delay 0.4 -max -clock DQS [get_ports DQ]
set_input_delay -0.4 -min -clock DQS [get_ports DQ]
# This is with respect to clock rising edge (default).

# Similarly for falling edge:
set_input_delay 0.35 -max -clock DQS -clock_fall \
  [get_ports DQ]
set_input_delay -0.35 -min -clock DQS -clock_fall \
  [get_ports DQ]
# The launch and capture are on the same edge:
set_multicycle_path 0 -setup -to UFF0/D
set_multicycle_path 0 -setup -to UFF5/D
```

The input delays represent the difference between the *DQ* and the *DQS* edges at the pins of the DUA. Even though these are normally launched together from the memory, there is a tolerance in the timing based upon the memory specification. Thus, the controller design within the DUA should consider that both signals can have skew with respect to each other. Here are the setup path reports to the two flip-flops. Assume that the setup requirement of the capturing flip-flop is 0.05ns and the hold requirement is

0.03ns. The DLL delay is assumed to be set to 1.25ns, a quarter of the cycle period.

Startpoint: DQ (**input port** clocked by DQS)
Endpoint: UFF0 (**rising edge-triggered flip-flop** clocked by DQS)
Path Group: DQS
Path Type: **max**

Point	Incr	Path

clock DQS (rise edge)	0.00	0.00
clock network delay (propagated)	0.00	0.00
input external delay	0.40	0.40 f
DQ (in)	0.00	0.40 f
UFF0/D (DF)	0.00	0.40 f
data arrival time		0.40
clock DQS (rise edge)	0.00	0.00
clock source latency	0.00	0.00
DQS (in)	0.00	0.00 r
UDLL0/Z (DLL)	1.25	1.25 r
UFF0/CP (DF)	0.00	1.25 r
library setup time	-0.05	1.20
data required time		1.20

data required time		1.20
data arrival time		-0.40

slack (MET)		0.80

Startpoint: DQ (**input port** clocked by DQS)
Endpoint: UFF5 (**falling edge-triggered flip-flop** clocked by DQS)
Path Group: DQS
Path Type: **max**

Point	Incr	Path

clock DQS (fall edge)	2.50	2.50
clock network delay (propagated)	0.00	2.50
input external delay	0.35	2.85 r
DQ (in)	0.00	2.85 r

UFF5/D (DFN)	0.00	2.85 r
data arrival time		2.85
clock DQS (fall edge)	2.50	2.50
clock source latency	0.00	2.50
DQS (in)	0.00	2.50 f
UDLL0/Z (DLL)	1.25	3.76 f
UFF5/CPN (DFN)	0.00	3.76 f
library setup time	-0.05	3.71
data required time		3.71

data required time		3.71
data arrival time		-2.85

slack (MET)		0.86

Here are the hold timing reports.

Startpoint: DQ (**input port** clocked by DQS)
 Endpoint: UFF0 (**rising edge-triggered flip-flop** clocked by DQS)
 Path Group: DQS
 Path Type: **min**

Point	Incr	Path

clock DQS (rise edge)	5.00	5.00
clock network delay (propagated)	0.00	5.00
input external delay	-0.40	4.60 r
DQ (in)	0.00	4.60 r
UFF0/D (DF)	0.00	4.60 r
data arrival time		4.60
clock DQS (fall edge)	2.50	2.50
clock source latency	0.00	2.50
DQS (in)	0.00	2.50 f
UDLL0/Z (DLL)	1.25	3.75 f
UFF0/CP (DF)	0.00	3.75 f
library hold time	0.03	3.78
data required time		3.78

data required time		3.78

data arrival time	-4.60

slack (MET)	0.82

Startpoint: DQ (**input port** clocked by DQS)
Endpoint: UFF5 (**falling edge-triggered flip-flop** clocked by DQS)
Path Group: DQS
Path Type: **min**

Point	Incr	Path

clock DQS (fall edge)	2.50	2.50
clock network delay (propagated)	0.00	2.50
input external delay	-0.35	2.15 f
DQ (in)	0.00	2.15 f
UFF5/D (DFN)	0.00	2.15 f
data arrival time		2.15
clock DQS (rise edge)	0.00	0.00
clock source latency	0.00	0.00
DQS (in)	0.00	0.00 r
UDLL0/Z (DLL)	1.25	1.25 r
UFF5/CPN (DFN)	0.00	1.25 r
library hold time	0.03	1.28
data required time		1.28

data required time		1.28
data arrival time		-2.15

slack (MET)		0.87

9.3.2 Write Cycle

In a write cycle, the *DQS* edges are quarter-cycle offset from the *DQ* signals coming out of the memory controller within the DUA so that the *DQS* strobe can be used to capture the data at the memory.

Figure 9-13 shows the required waveforms at the memory pins. The *DQS* signal must be aligned to be at the center of the *DQ* window at the memory

pins. Note that aligning DQ and DQS to have the same property at the memory controller (inside DUA) is not enough for these signals to have the required alignment at the SDRAM pins due to mismatch in the IO buffer delays or variations in PCB interconnect traces. Thus, the DUA typically provides additional DLL controls in the write cycle to achieve the required quarter cycle offset between the DQS and the DQ signals.

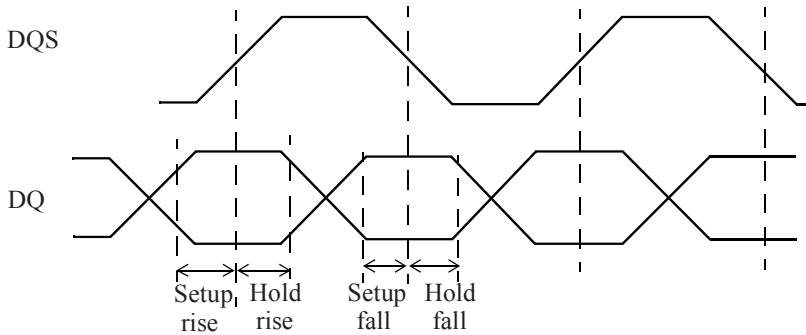


Figure 9-13 *DQ and DQS signals at memory pins during DDR write cycle.*

Constraining the outputs for this mode depends on how the clocks are generated in the controller. We consider two cases.

Case 1: Internal 2x Clock

If an internal clock that is double the frequency of the DDR clock is available, the output logic can be similar to the one shown in Figure 9-14. The DLL provides a mechanism to skew the DQS clock if necessary so that the setup and hold requirements at the memory pins are met. In some cases, the DLL may not be used - instead a negative edge flip-flop is used to get the 90 degree offset.

For the scenario shown in Figure 9-14, the outputs can be constrained as:

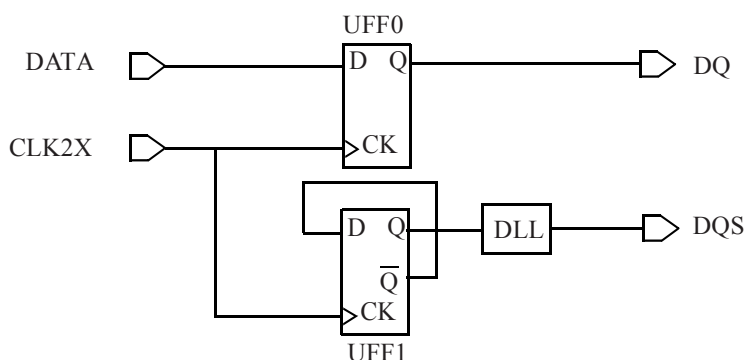


Figure 9-14 *DQS is a divide-by-2 of internal clock.*

```
# 166MHz (333Mbps) DDR; 2x clock is at 333MHz:
create_clock -period 3 [get_ports CLK2X]

# Define a 1x generated clock at the output of flip-flop:
create_generated_clock -name pre_DQS -source CLK2X \
  -divide_by 2 [get_pins UFF1/Q]
# Create the delayed version as DQS assuming 1.5ns DLL delay:
create_generated_clock -name DQS -source UFF1/Q \
  -edges {1 2 3} -edge_shift {1.5 1.5 1.5} [get_ports DQS]
```

The timing at the *DQ* output pins has to be constrained with respect to the generated clock *DQS*.

Assume that the setup requirements between the *DQ* and *DQS* pins at the DDR SDRAM are 0.25ns and 0.4ns for the rising edge and falling edge of *DQ* respectively. Similarly, a hold requirement of 0.15ns and 0.2ns is assumed for the rising and falling edges of *DQ* pins. The DLL delay at the *DQS* output has been set to a quarter-cycle, which is 1.5ns. The waveforms are shown in Figure 9-15.

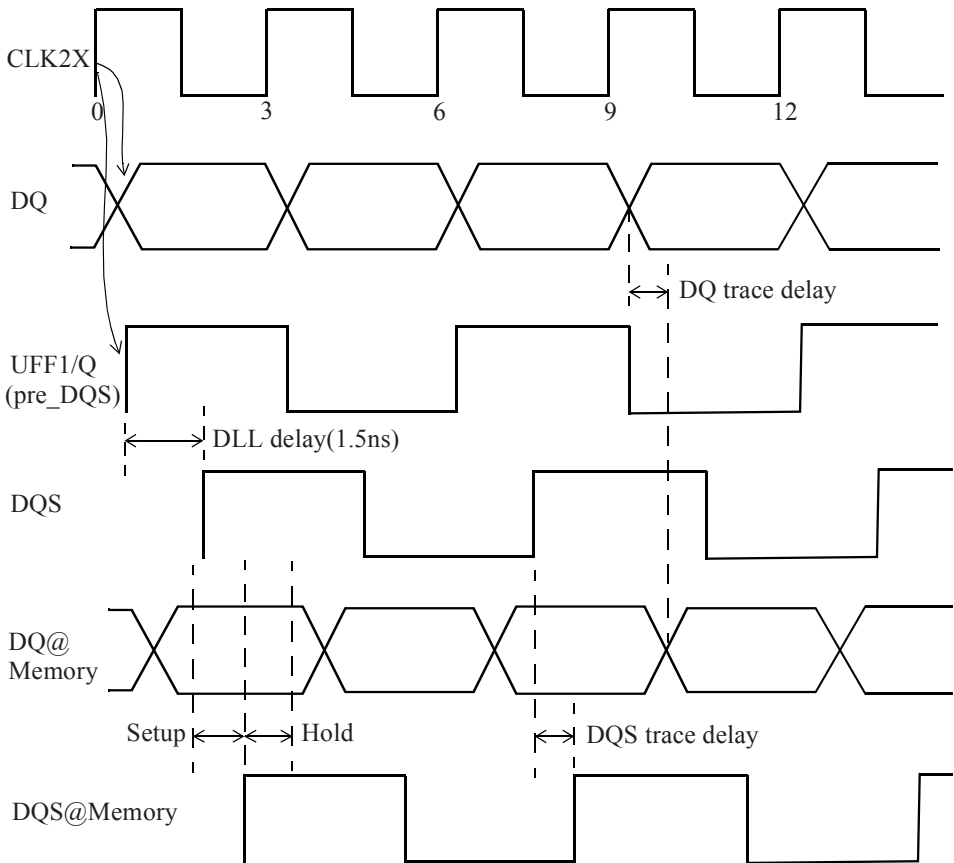


Figure 9-15 *DQS and DQ signals obtained from internal 2x clock.*

```
set_output_delay -clock DQS -max 0.25 -rise [get_ports DQ]
# Default above is rising clock.
set_output_delay -clock DQS -max 0.4 -fall [get_ports DQ]
# If setup requirements are different for falling edge of DQS,
# that can be specified by using the -clock_fall option.
```

```
set_output_delay -clock DQS -min -0.15 -rise DQ
set_output_delay -clock DQS -min -0.2 -fall DQ
```

Here is the setup report through the output *DQ*. The setup check is from the rise edge of *CLK2X* at 0ns which launches *DQ* to the rise edge of *DQS* at 1.5ns.

```
Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLK2X)
Endpoint: DQ (output port clocked by DQS)
Path Group: DQS
Path Type: max
```

Point	Incr	Path

clock CLK2X (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLK2X (in)	0.00	0.00 r
UFF0/CP (DFD1)	0.00	0.00 r
UFF0/Q (DFD1)	0.12	0.12 f
DQ (out)	0.00	0.12 f
data arrival time		0.12
clock DQS (rise edge)	1.50	1.50
clock CLK2X (source latency)	0.00	1.50
CLK2X (in)	0.00	1.50 r
UFF1/Q (DFD1) (gclock source)	0.12	1.62 r
UDLL0/Z (DLL)	0.00	1.62 r
DQS (out)	0.00	1.62 r
output external delay	-0.40	1.22
data required time		1.22

data required time		1.22
data arrival time		-0.12

slack (MET)		1.10

Note that the quarter-cycle delay in the above report appears in the first line with the *DQS* clock edge rather than on the line showing the DLL instance *UDLL0*. This is because the DLL delay has been modeled as part of

the generated clock definition for *DQS* instead of in the timing arc for the DLL.

Here is the hold report through output *DQ*. The hold check is done from the rising edge of clock *CLK2X* which launches *DQ* at 3ns to the previous rising edge of *DQS* at 1.5ns.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLK2X)

Endpoint: DQ (output port clocked by DQS)

Path Group: DQS

Path Type: min

Point	Incr	Path

clock CLK2X (rise edge)	3.00	3.00
clock source latency	0.00	3.00
CLK2X (in)	0.00	3.00 r
UFF0/CP (DFD1)	0.00	3.00 r
UFF0/Q (DFD1)	0.12	3.12 f
DQ (out)	0.00	3.12 f
data arrival time		3.12
 clock DQS (rise edge)	 1.50	 1.50
clock CLK2X (source latency)	0.00	1.50
CLK2X (in)	0.00	1.50 r
UFF1/Q (DFD1) (gclock source)	0.12	1.62 r
UDLL0/Z (DLL)	0.00	1.62 r
DQS (out)	0.00	1.62 r
output external delay	0.20	1.82
data required time		1.82

data required time		1.82
data arrival time		-3.12

slack (MET)		1.30

Case 2: Internal 1x Clock

When only an internal 1x clock is available, the output circuitry may typically be similar to that shown in Figure 9-16.

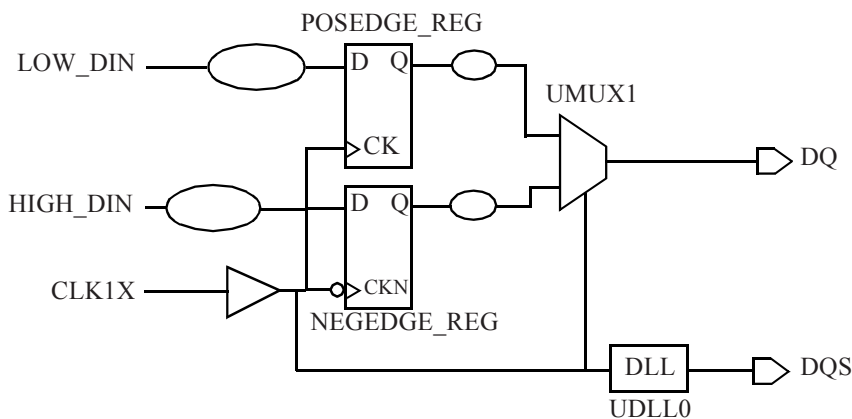


Figure 9-16 Output logic in DUA using internal 1x clock.

There are two flip-flops used to generate the *DQ* data. The first flip-flop *NEGEDGE_REG* is triggered by the negative edge of the clock *CLK1X*, and the second flip-flop *POSEDGE_REG* is triggered by the positive edge of the clock *CLK1X*. Each flip-flop latches the appropriate edge data and this data is then multiplexed out using the *CLK1X* as the multiplexer select. When *CLK1X* is high, the output of flip-flop *NEGEDGE_REG* is sent to *DQ*. When *CLK1X* is low, the output of flip-flop *POSEDGE_REG* is sent to *DQ*. Hence, data arrives at the output *DQ* on both edges of clock *CLK1X*. Notice that each flip-flop has half a cycle to propagate data to the input of the multiplexer so that the input data is ready at the multiplexer before it is selected by the *CLK1X* edge. The relevant waveforms are shown in Figure 9-17.

```
# Create the 1x clock:
```

```
create_clock -name CLK1X -period 6 [get_ports CLK1X]
```

```
# Define a generated clock at DQS. It is a divide-by-1 of
# CLK1X. Assume a quarter-cycle delay of 1.5ns on UDLL0:
create_generated_clock -name DQS -source CLK1X \
-edges {1 2 3} -edge_shift {1.5 1.5 1.5} [get_ports DQS]

# Define a setup check of 0.25 and 0.3 between DQ and DQS
# pins on rising and falling edge of clock:
set_output_delay -max 0.25 -clock DQS [get_ports DQ]
set_output_delay -max 0.3 -clock DQS -clock_fall \
[get_ports DQ]
set_output_delay -min -0.2 -clock DQS [get_ports DQ]
set_output_delay -min -0.27 -clock DQS -clock_fall \
[get_ports DQ]
```

The setup and hold checks verify the timing from the multiplexer to the output. One of the setup checks is from the rising edge of *CLK1X* at the multiplexer input (which launches the *NEGEDGE_REG* data) to the rising edge of *DQS*. The other setup check is from the falling edge of *CLK1X* at the multiplexer input (which launches the *POSEDGE_REG* data) to the falling edge of *DQS*. Similarly, the hold checks are from the same *CLK1X* edges (as for setup checks) to the previous falling or rising edges of *DQS*.

Here is the setup timing check report through the port *DQ*. The check is between the rising edge of *CLK1X*, which selects the *NEGEDGE_REG* output, and the rising edge of the *DQS*.

```
Startpoint: CLK1X (clock source 'CLK1X')
Endpoint: DQ (output port clocked by DQS)
Path Group: DQS
Path Type: max
```

Point	Incr	Path

clock CLK1X (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLK1X (in)	0.00	0.00 r
UMUX1/S (MUX2D1) <-	0.00	0.00 r
UMUX1/Z (MUX2D1)	0.07	0.07 r

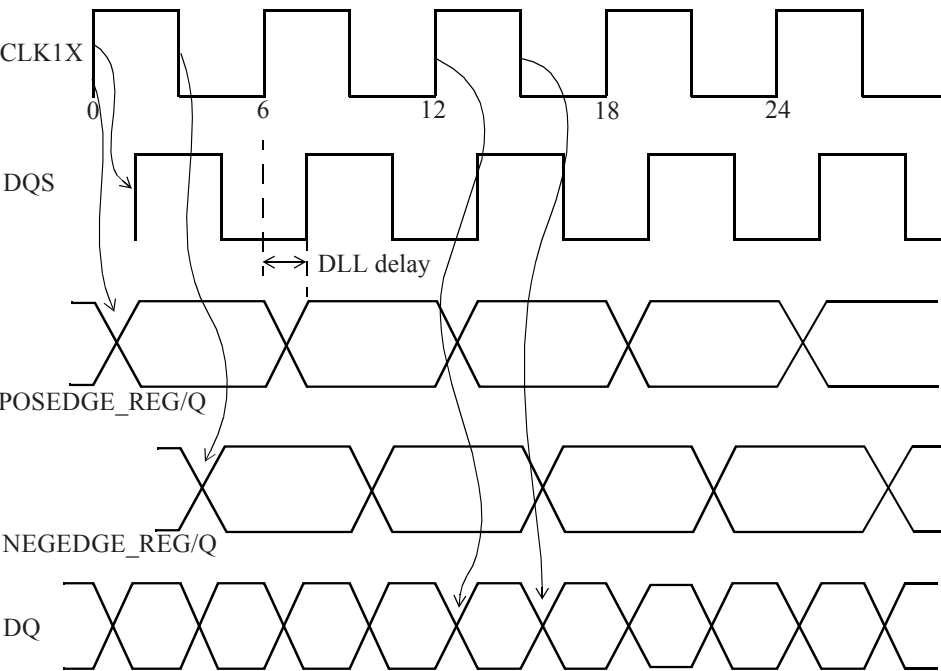


Figure 9-17 *DQS and DQ signals obtained using 1x internal clock.*

DQ (out)	0.00	0.07 r
data arrival time		0.07
clock DQS (rise edge)	1.50	1.50
clock CLK1X (source latency)	0.00	1.50
CLK1X (in)	0.00	1.50 r
UBUF0/Z (BUFFD1)	0.03	1.53 r
UDLL0/Z (DLL)	0.00	1.53 r
DQS (out)	0.00	1.53 r
output external delay	-0.25	1.28
data required time		1.28

data required time		1.28
data arrival time		-0.07

slack (MET)

1.21

Here is another setup timing check report through the port *DQ*. This setup check is between the falling edge of *CLK1X* which selects the *POSEDGE_REG* output and the falling edge of the *DQS*.

Startpoint: CLK1X (clock source 'CLK1X')
 Endpoint: DQ (output port clocked by DQS)
 Path Group: DQS
 Path Type: max

Point	Incr	Path

clock CLK1X (fall edge)	3.00	3.00
clock source latency	0.00	3.00
CLK1X (in)	0.00	3.00 f
UMUX1/S (MUX2D1) <-	0.00	3.00 f
UMUX1/Z (MUX2D1)	0.05	3.05 f
DQ (out)	0.00	3.05 f
data arrival time		3.05
 clock DQS (fall edge)	 4.50	 4.50
clock CLK1X (source latency)	0.00	4.50
CLK1X (in)	0.00	4.50 f
UBUF0/Z (BUFFD1)	0.04	4.54 f
UDLL0/Z (DLL)	0.00	4.54 f
DQS (out)	0.00	4.54 f
output external delay	-0.30	4.24
data required time		4.24

data required time		4.24
data arrival time		-3.05

slack (MET)		1.19

Here is the hold timing check report through the port *DQ*. The check is between the rising edge of *CLK1X* and the previous falling edge of *DQS*.

Startpoint: CLK1X (clock source 'CLK1X')
Endpoint: DQ (output port clocked by DQS)
Path Group: DQS
Path Type: min

Point	Incr	Path

clock CLK1X (rise edge)	6.00	6.00
clock source latency	0.00	6.00
CLK1X (in)	0.00	6.00 r
UMUX1/S (MUX2D1) <-	0.00	6.00 r
UMUX1/Z (MUX2D1)	0.05	6.05 f
DQ (out)	0.00	6.05 f
data arrival time		6.05
 clock DQS (fall edge)	 4.50	 4.50
clock CLK1X (source latency)	0.00	4.50
CLK1X (in)	0.00	4.50 f
UBUF0/Z (BUFFD1)	0.04	4.54 f
UDLL0/Z (DLL)	0.00	4.54 f
DQS (out)	0.00	4.54 f
output external delay	0.27	4.81
data required time		4.81

data required time		4.81
data arrival time		-6.05

slack (MET)		1.24

Here is another hold timing check report through the port *DQ*. This check is between the falling edge of *CLK1X* and the previous rising edge of *DQS*.

Startpoint: CLK1X (clock source 'CLK1X')
Endpoint: DQ (output port clocked by DQS)
Path Group: DQS
Path Type: min

Point	Incr	Path

clock CLK1X (fall edge)	3.00	3.00
clock source latency	0.00	3.00
CLK1X (in)	0.00	3.00 f
UMUX1/S (MUX2D1) <-	0.00	3.00 f
UMUX1/Z (MUX2D1)	0.05	3.05 f
DQ (out)	0.00	3.05 f
data arrival time		3.05
clock DQS (rise edge)	1.50	1.50
clock CLK1X (source latency)	0.00	1.50
CLK1X (in)	0.00	1.50 r
UBUF0/Z (BUFFD1)	0.03	1.53 r
UDLL0/Z (DLL)	0.00	1.53 r
DQS (out)	0.00	1.53 r
output external delay	0.20	1.73
data required time		1.73

data required time		1.73
data arrival time		-3.05

slack (MET)		1.32

While the above interface timing analysis has ignored the effect of any loads on the outputs, additional load can be specified (using *set_load*) for more accuracy. However, STA can be supplemented with circuit simulation for achieving a robust DRAM timing as described below.

The *DQ* and the *DQS* signals for the DDR interface typically use ODT¹ in read and write modes to reduce any reflections due to impedance mismatch at the DRAM and at the DUA. The timing models used for STA are not able to provide adequate accuracy in presence of ODT termination. The designer may use an alternate mechanism such as detailed circuit level simulation to validate the signal integrity and the timing of the DRAM interface.

1. On-Die Termination.

9.4 Interface to a Video DAC

Consider Figure 9-18 which shows a typical DAC¹ interface where a high-speed clock is transferring data to the slow-speed clock interface of the DAC.

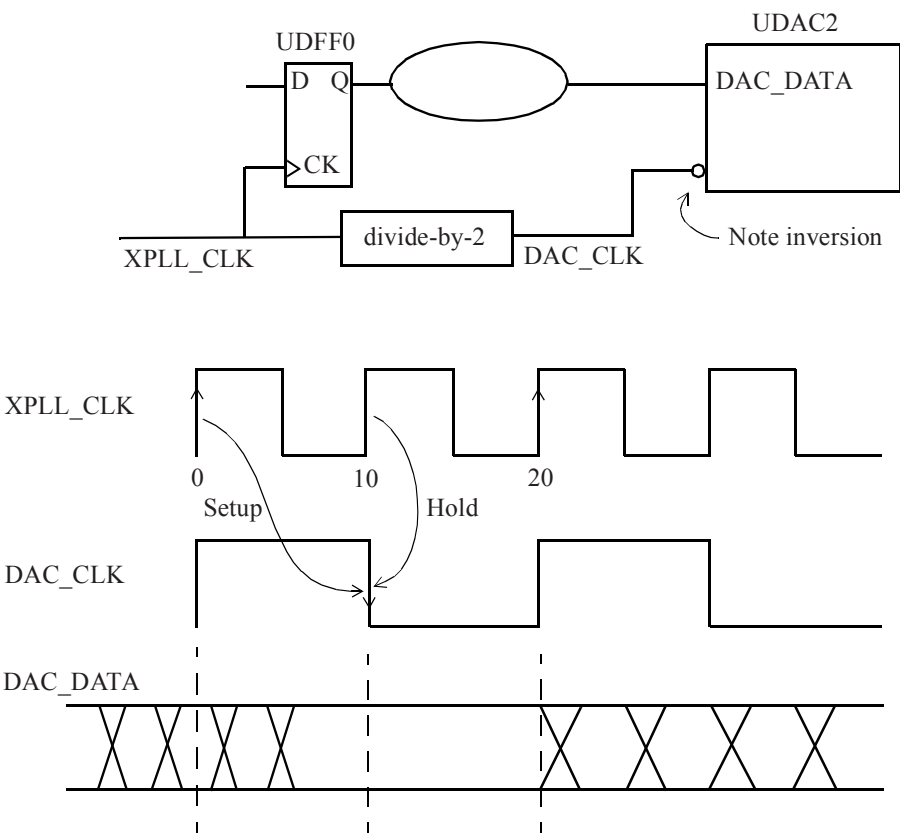


Figure 9-18 *Video DAC interface.*

1. Digital to Analog Converter.

Clock *DAC_CLK* is a divide-by-2 of the clock *XPLL_CLK*. The DAC setup and hold checks are with respect to the falling edge of *DAC_CLK*.

In this case, the setup time is considered as a single cycle (*XPLL_CLK*) path, even though the interface from a faster clock domain to a slower clock domain can be specified as a multicycle path if necessary. As shown in Figure 9-18, the rising edge of *XPLL_CLK* launches the data and the falling edge of *DAC_CLK* captures the data. Here is the setup report.

Startpoint: UDFF0

(rising edge-triggered flip-flop clocked by XPLL_CLK)

Endpoint: UDAC2

(falling edge-triggered flip-flop clocked by DAC_CLK)

Path Group: DAC_CLK

Path Type: max

Point	Incr	Path

clock XPLL_CLK (rise edge)	0.00	0.00
clock source latency	0.00	0.00
XPLL_CLK (in)	0.00	0.00 r
UDFF0/CK (DF)	0.00	0.00 r
UDFF0/Q (DF)	0.12	0.12 f
UBUF0/Z (BUFF)	0.06	0.18 f
UDFF2/D (DFN)	0.00	0.18 f
data arrival time		0.18
 clock DAC_CLK (fall edge)	 10.00	 10.00
clock XPLL_CLK (source latency)	0.00	10.00
XPLL_CLK (in)	0.00	10.00 r
UDFF1/Q (DF) (gclock source)	0.13	10.13 f
UDAC2/CKN (DAC)	0.00	10.13 f
library setup time	-0.04	10.08
data required time		10.08

data required time		10.08
data arrival time		-0.18

slack (MET)		9.90

Note that the interface is from a faster clock to a slower clock, thus it can be made into a two-cycle path if necessary.

Here is the hold timing report.

Startpoint: UDFF0
 (rising edge-triggered flip-flop clocked by XPLL_CLK)
Endpoint: UDAC2
 (falling edge-triggered flip-flop clocked by DAC_CLK)
Path Group: DAC_CLK
Path Type: min

Point	Incr	Path

clock XPLL_CLK (rise edge)	10.00	10.00
clock source latency	0.00	10.00
XPLL_CLK (in)	0.00	10.00 r
UDFF0/CK (DF)	0.00	10.00 r
UDFF0/Q (DF)	0.12	10.12 r
UBUF0/Z (BUFF)	0.05	10.17 r
UDFF2/D (DFN)	0.00	10.17 r
data arrival time		10.17
clock DAC_CLK (fall edge)	10.00	10.00
clock XPLL_CLK (source latency)	0.00	10.00
XPLL_CLK (in)	0.00	10.00 r
UDFF1/Q (DF) (gclock source)	0.13	10.13 f
UDAC2/CKN (DAC)	0.00	10.13 f
library hold time	0.03	10.16
data required time		10.16

data required time		10.16
data arrival time		-10.17

slack (MET)		0.01

The hold check is done one cycle prior to the setup capture edge. In this case, the most critical hold check is the one where the setup and launch edges are the same, and this is shown in the hold timing report.

